



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №4
Технології розробки програмного забезпечення
«ШАБЛОНИ «SINGLETON»,
«ITERATOR», «PROXY», «STATE»,
«STRATEGY»»

Виконав:
студент групи ІА-24
Іскандаров Е. Е

Перевірив:
Мягкий М. Ю.

Київ 2025

Тема: ШАБЛОНИ «SINGLETON», «ITERATOR», «PROXY», «STATE», «STRATEGY».

Зміст

Короткі теоретичні відомості.....	2
Реалізація шаблону проектування	3
Структура шаблону	6
Результат виконання	7

Короткі теоретичні відомості

Шаблон проектування — це формалізований опис типового рішення, що часто використовується при проектуванні інформаційних систем, має загальноживану назву та рекомендації для застосування в різних ситуаціях. Його основні переваги включають полегшення створення структури системи, виділення значимих елементів та зв'язків, що робить модель більш зрозумілою та простою для вивчення. Шаблони підвищують стійкість системи до змін, спрощують її розширення та вдосконалення. Вони представляють собою ескізи архітектурних рішень, які зручно застосовувати у відповідних обставинах.

Розглянемо шаблони, надані для лабораторної роботи

Шаблон Singleton гарантує створення лише одного екземпляра класу з глобальним доступом до нього, що є зручним для зберігання загального стану, як-от конфігурації чи доступ до бази даних. Програма завжди звертається до єдиного доступного екземпляра, забезпечуючи його унікальність і захист від випадкових змін.

Шаблон Iterator полегшує роботу з колекціями, дозволяючи послідовно отримувати доступ до їхніх елементів, не розкриваючи внутрішньої структури. Це особливо зручно для перебору великих масивів або списків, адже ітератор забезпечує єдиний інтерфейс доступу незалежно від способу реалізації колекції.

Шаблон Proxy створює об'єкт-замісник, який контролює доступ до основного об'єкта, що є корисним для обмеження або розширення функціональності за допомогою додаткової логіки, наприклад, безпеки чи кешування. Замісник перехоплює виклики до основного об'єкта, виконує додаткові дії і потім передає запит далі.

Шаблон State дозволяє керувати поведінкою об'єкта залежно від його поточного стану. Це актуально, коли об'єкт може перебувати в різних станах, кожен з яких змінює його поведінку.

Шаблон Strategy забезпечує можливість вибору різних алгоритмів для виконання певного завдання в процесі роботи програми. Це дозволяє об'єкту змінювати свою поведінку залежно від зовнішніх умов або потреб користувача без зміни

основного коду. Кожен алгоритм оформлений як окремий клас, і об'єкт використовує потрібну стратегію відповідно до ситуації, роблячи код гнучким і масштабованим.

Хід роботи:

Тема:

..6 Web-browser (proxy, chain of responsibility, factory method, template method, visitor, p2p)

Веб-браузер повинен мати можливість зробити наступне: мати адресний рядок для введення адреси сайту, переміщатися і відображати структуру html документа, переглядати підключений javascript та css файли, перегляд всіх підключених ресурсів (зображень), коректна обробка відповідей з сервера (коди відповідей HTTP) - переходи при перенаправленнях, відображення сторінок 404 і 502/503.

Для реалізації програми WebBrowser доцільно використати шаблон проектування Проксу. Цей шаблон дозволяє створити гнучку систему, в якій запити до веб-ресурсів можуть бути оброблені через проміжний об'єкт, що додає додаткову функціональність, таку як перевірка, кешування або логування, без зміни основної логіки клієнтського коду.

У випадку з обробкою веб-запитів важливо забезпечити можливість динамічного додавання поведінки (наприклад, перевірка валідності URL, обробка помилок HTTP, перенаправлення), що дає змогу адаптувати програму до різних вимог. Завдяки використанню Проксу кожна додаткова функція може бути інкапсульована в проміжному класі, забезпечуючи контроль доступу та оптимізацію без змін у реалізації реального об'єкта.

Використання Проксу дозволяє уникнути перевантаження основного класу складною логікою та спрощує розширення програми. Наприклад, можна легко додати нові функції, такі як моніторинг запитів або перевірка авторизації, лише шляхом зміни або розширення проксі-класу, не змінюючи базову логіку обробки запитів. Це робить програму більш масштабованою, легкою для тестування та підтримки.

```
3 fx interface WebRequest { no usages 2 implementations
4 fx     void fetch(String url); no usages 2 implementations
5     }
```

Рис. 1 Код інтерфейсу WebRequest

Інтерфейс WebRequest:

- Визначає метод `fetch(String url)` для виконання HTTP-запиту.

```

3      class RealWebRequest implements WebRequest { 2 usages
4          @Override 7 usages
5          public void fetch(String url) {
6              System.out.println("Fetching URL: " + url);
7
8              // Емуляція отримання відповіді від сервера
9              if (url.contains("redirect")) {
10                  System.out.println("HTTP 302: Redirecting to new location...");
11                  fetch(url.replace( target: "redirect", replacement: "final"));
12              } else if (url.contains("404")) {
13                  System.out.println("HTTP 404: Page not found.");
14              } else if (url.contains("503")) {
15                  System.out.println("HTTP 503: Service unavailable.");
16              } else {
17                  System.out.println("HTTP 200: Successfully fetched content from " + url);
18              }
19          }
20      }
21

```

Рис 2. Код класу RealWebRequest

Клас RealWebRequest:

- Реалізує логіку виконання реальних запитів.
- Обробляє різні HTTP-коди, такі як 302, 404, 503.

```

3  class WebRequestProxy implements WebRequest { 1 usage
4      private RealWebRequest realWebRequest; 2 usages
5
6      public WebRequestProxy() { 1 usage
7          this.realWebRequest = new RealWebRequest();
8      }
9
10     @Override 7 usages
11     public void fetch(String url) {
12         System.out.println("[Proxy] Validating URL...");
13
14         if (!isValidUrl(url)) {
15             System.out.println("Invalid URL: " + url);
16             return;
17         }
18
19         System.out.println("[Proxy] URL is valid. Proceeding with request.");
20
21         realWebRequest.fetch(url);
22     }
23
24     private boolean isValidUrl(String url) { 1 usage
25         return url.startsWith("http://") || url.startsWith("https://");
26     }
27 }

```

Рис. 3 Код класу WebRequestProxy

Клас WebRequestProxy:

- Додає додаткову логіку, наприклад, перевірку валідності URL.
- Перехоплює виклики до RealWebRequest.

```

3  public class ProxyDemo {
4      public static void main(String[] args) {
5          WebRequest webRequest = new WebRequestProxy();
6
7          webRequest.fetch(url: "http://example.com");
8          webRequest.fetch(url: "http://example.com/redirect");
9          webRequest.fetch(url: "http://example.com/404");
10         webRequest.fetch(url: "http://example.com/503");
11         webRequest.fetch(url: "invalid_url");
12     }
13 }

```

Рис 4. Код класу ProxyDemo

Головний клас ProxyDemo:

- Демонструє використання проксі для обробки запитів.

При виконанні `webRequest.fetch(url)` проксі спочатку перевіряє валідність URL. У випадку валідного URL проксі передає запит до реального об'єкта. Реальний об'єкт виконує основну роботу, включаючи обробку перенаправлень і помилок.

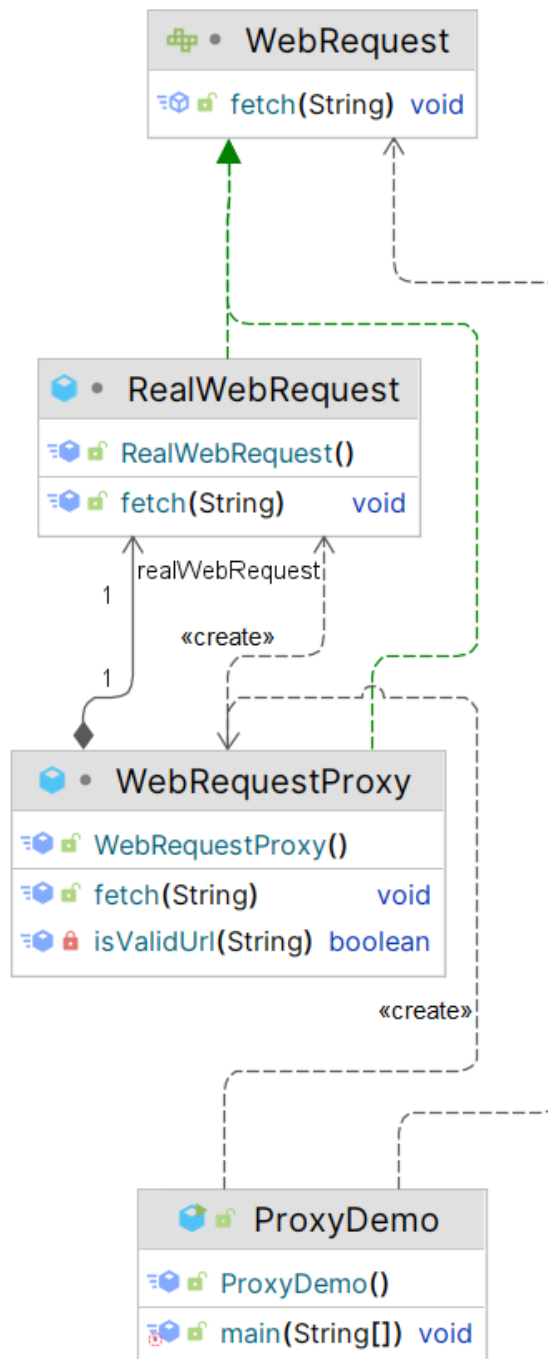


Рис 4. Структура шаблону Proxy

Шаблон **Proxy**, використаний у реалізації класів, що додають рівень абстракції між клієнтом і реальним об'єктом, є прикладом ефективного підходу до

розробки, орієнтованого на контроль доступу, оптимізацію продуктивності та підвищення безпеки. Цей шаблон дозволяє створити проміжний об'єкт, який контролює доступ до реального об'єкта, забезпечуючи додаткову логіку, наприклад, перевірку прав доступу, кешування або відкладене створення.

Завдяки Proxy можна уникнути прямої залежності клієнтського коду від складного або ресурсоемного об'єкта. Це підвищує гнучкість системи, оскільки дозволяє змінювати або розширювати поведінку реального об'єкта, не змінюючи основний клієнтський код. Крім того, Proxy значно спрощує тестування і підтримку коду, оскільки його легко адаптувати до нових вимог, таких як логування чи моніторинг.

Цей шаблон дозволяє побудувати систему, яка зберігає чітку структуру, забезпечує високу керованість, і водночас уникає складних розгалужень або дублювання коду, які могли б виникнути при менш структурованих підходах.

Результат виконання

```
[Proxy] Validating URL...
[Proxy] URL is valid. Proceeding with request.
Fetching URL: http://example.com
HTTP 200: Successfully fetched content from http://example.com
[Proxy] Validating URL...
[Proxy] URL is valid. Proceeding with request.
Fetching URL: http://example.com/redirect
HTTP 302: Redirecting to new location...
Fetching URL: http://example.com/final
HTTP 200: Successfully fetched content from http://example.com/final
[Proxy] Validating URL...
[Proxy] URL is valid. Proceeding with request.
Fetching URL: http://example.com/404
HTTP 404: Page not found.
```

Рис. 5. Результат виконання

Реалізація: https://github.com/Elmir2/trpz_labs

Висновок: У ході виконання даної лабораторної роботи було розглянуто структуру та призначення шаблонів проектування «SINGLETON», «ITERATOR», «PROXY», «STATE», «STRATEGY». Кожен з розглянутих шаблонів має свої переваги та недоліки. Далі було обрано шаблон, який підійде для реалізації майбутньої системи. Останнім етапом була реалізація шаблону та дослідження його структури, переваг у використанні для заданої системи.